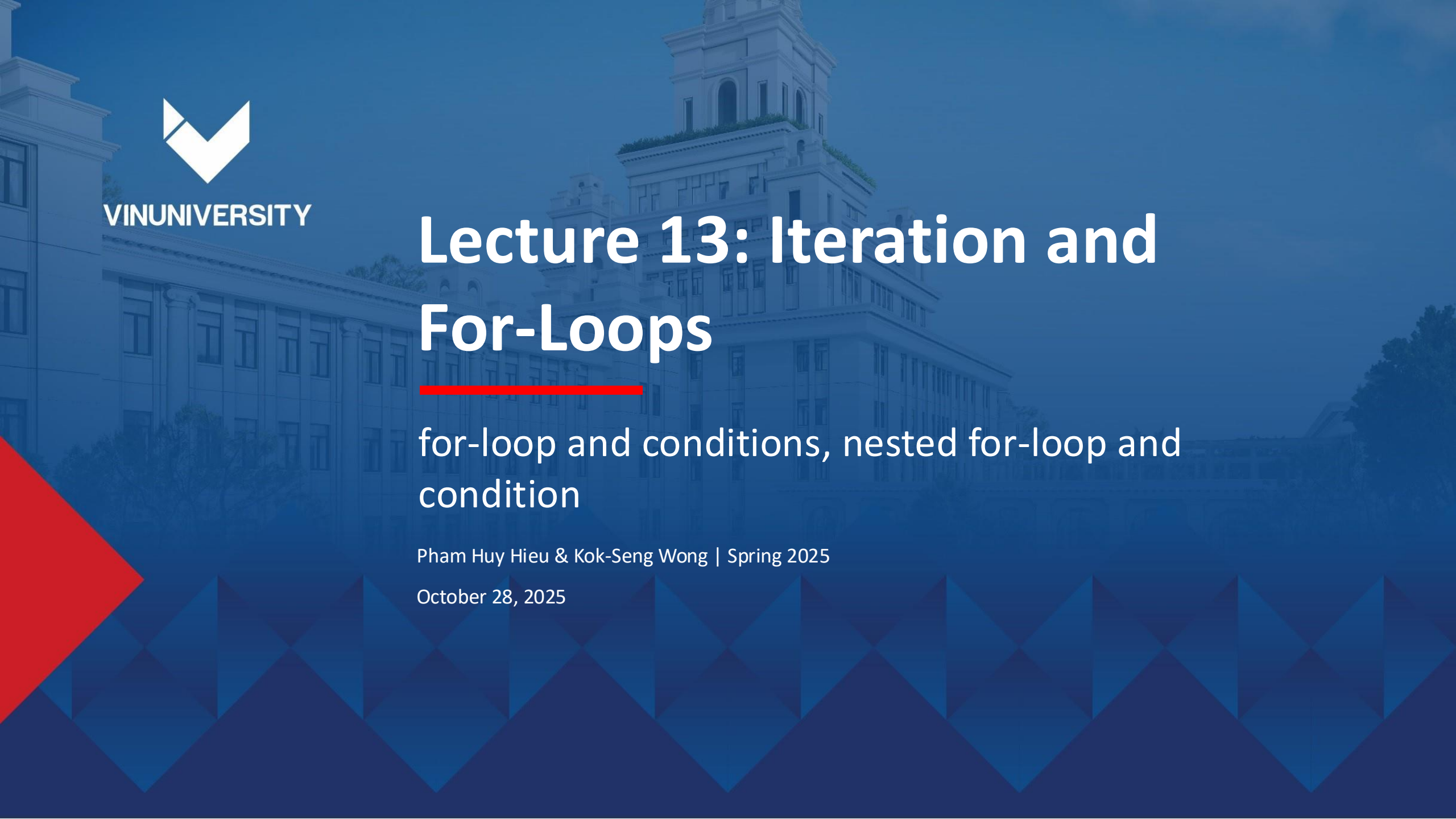


VINUNIVERSITY

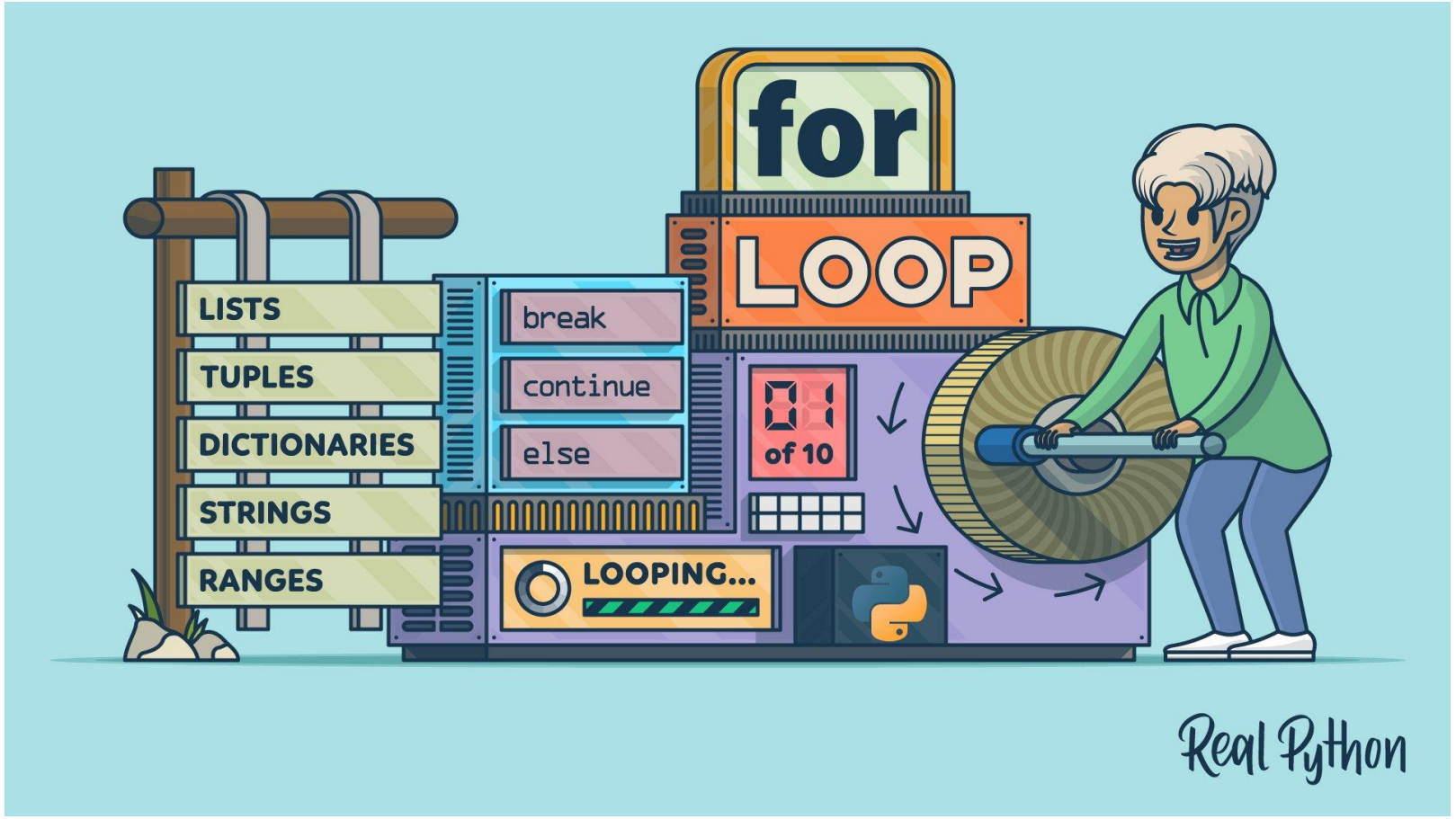
Lecture 13: Iteration and For-Loops

for-loop and conditions, nested for-loop and condition

Pham Huy Hieu & Kok-Seng Wong | Spring 2025

October 28, 2025

MOTIVATIONAL EXAMPLE



In Python, you'll generally use for **loops** when you need to iterate over the items in a data collection. This type of loop lets you traverse different data collections and run a specific group of statements on or with each item in the input collection (source).

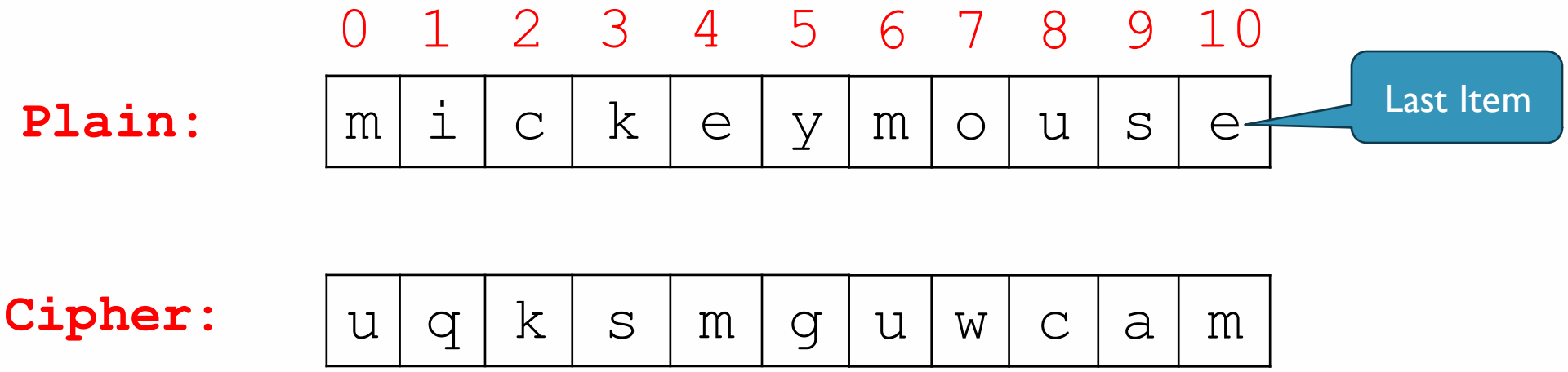
LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand the concept of iteration in Python.
- describe the syntax for for-loops.
- identify inner and outer loops in a nested for-loop.

ITERATION

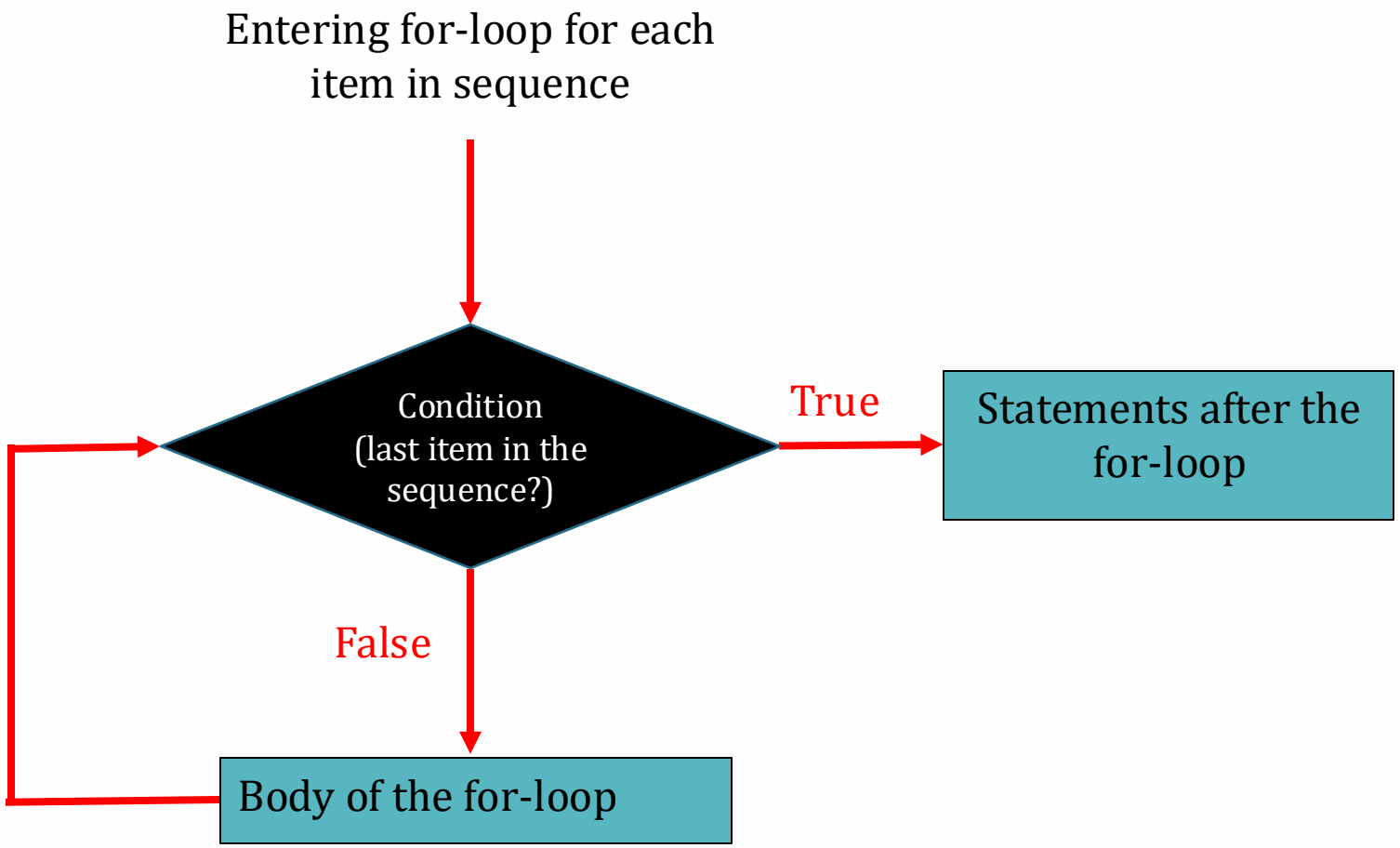
- Iteration is a basic building block of all programs is to be able to repeat some code over and over again.
- Iteration: A repetitive execution (until a condition is met).
- Example: Caesar Cipher (shift key = 8 position)



FOR-LOOP

- for-loop: used for iterating over a sequence (e.g., set, string, list, tuple, or dictionary).
- Definite iteration: do something a fixed number (pre-determined) of times.
- Processing each item in a sequence and execute a set of statements, once for each item in the sequence.

FOR-LOOP FLOWCHART

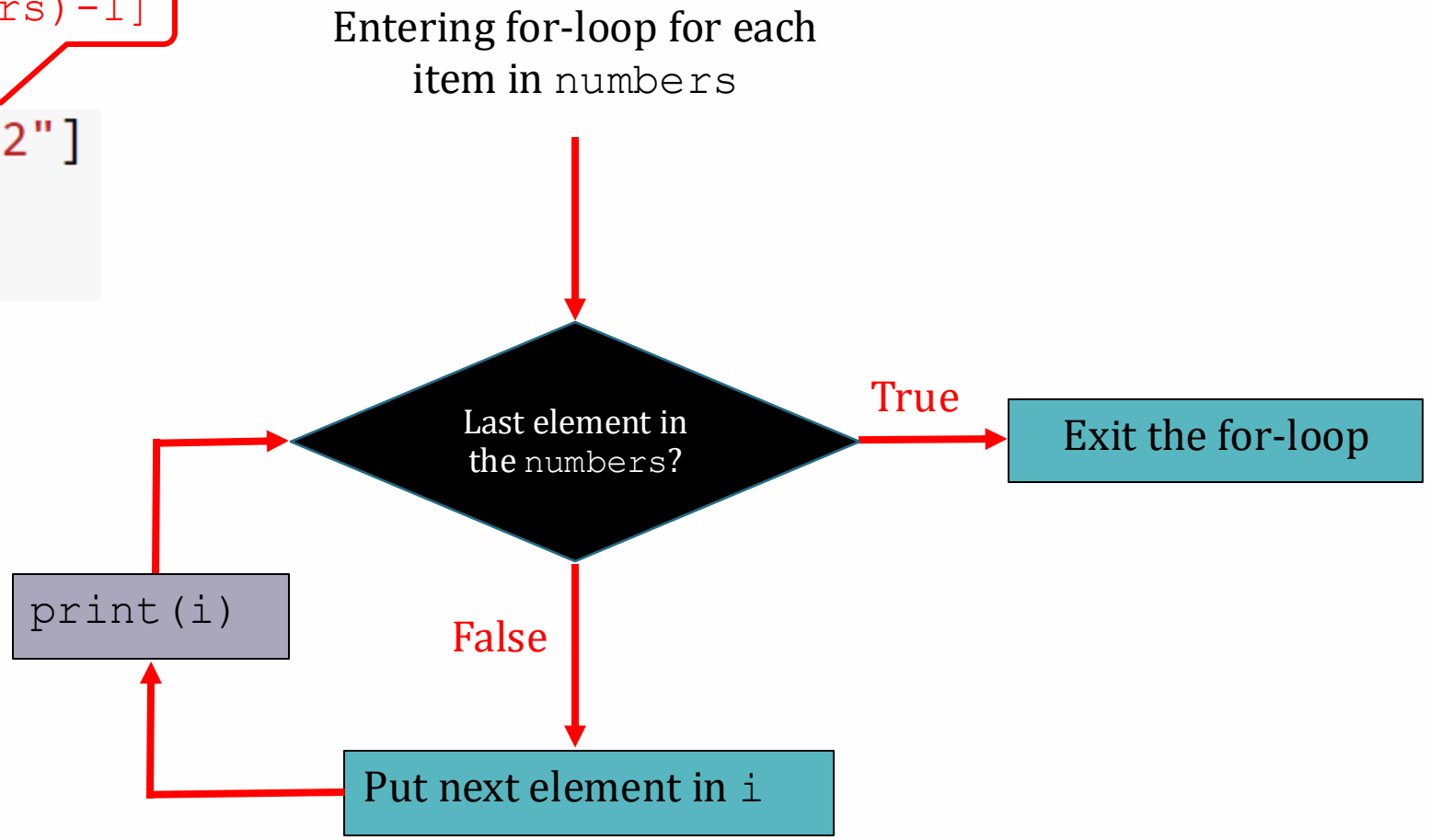


FOR-LOOP: PROCESSING LISTS

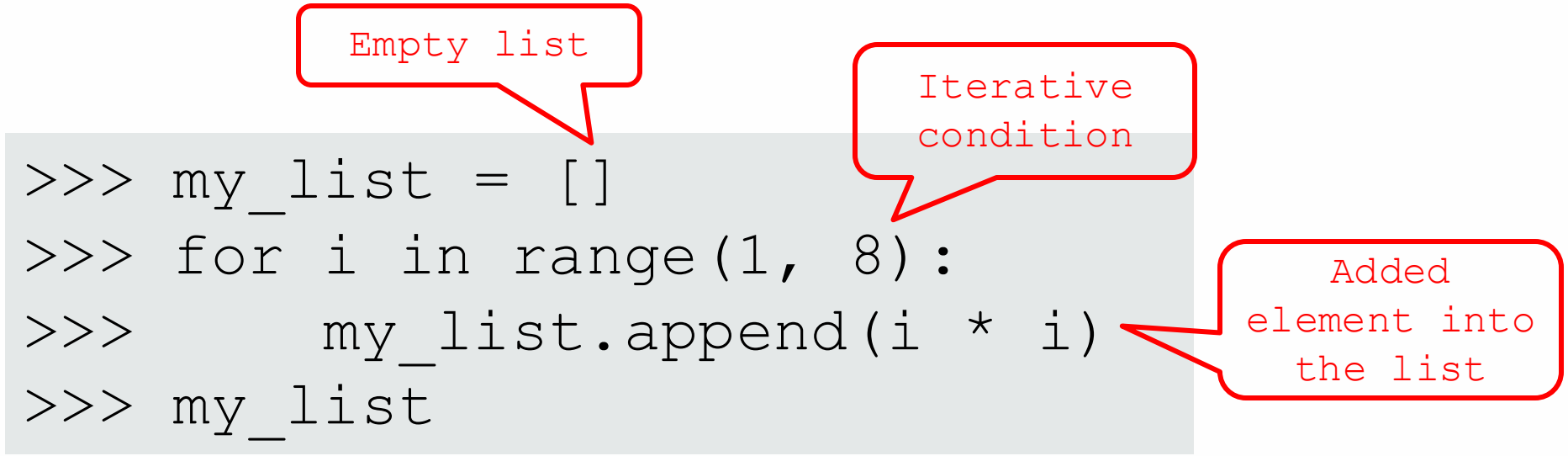
`numbers[len(numbers)-1]`

```
numbers = ["11", "12", "22"]
for i in numbers:
    print(i)
```

loop sequence: `numbers`
loop variable: `i`
loop body: `print(i)`



CREATE A LIST: USING FOR LOOPS



CLASS PRACTICE

Sample Code:

```
numbers = ["11", "12", "22"]
for i in numbers:
    print(      ?       )
```

Output:

```
11 12 22
```

CLASS PRACTICE

Sample Code:

```
x = ["a", "b", "c"]
```

Output:

```
Output 1 is a
Output 2 is b
Output 3 is c
```

BREAK AND CONTINUE

- `Break` and `continue` are two ways to modify the behavior of for-loop and while-loop (will be covered in Lecture 14).
- `Break`: causes the program to exit a loop early.
- `Continue`: causes the program to stop running code in a loop and start back at the top of the loop.

```
for-loop iteration:
    statement 1
    statement 2
    break
    ...
    statement n
statement(s)
```



```
for-loop iteration:
    statement 1
    statement 2
    continue
    ...
    statement n
statement(s)
```

BREAK - EXAMPLE

- Break: causes the program to exit a loop early.

```
for i in range(10):
    if i >= 3:
        print(i)
        break
        print(i + 1)
print("[for-loop exited!]")
```

```
for i in range(10):
    print(i)
    if i >= 3:
        break
print('for-loop exited!')
```

CONTINUE - EXAMPLE

- `Continue`: causes the program to stop running code in a loop and start back at the top of the loop.

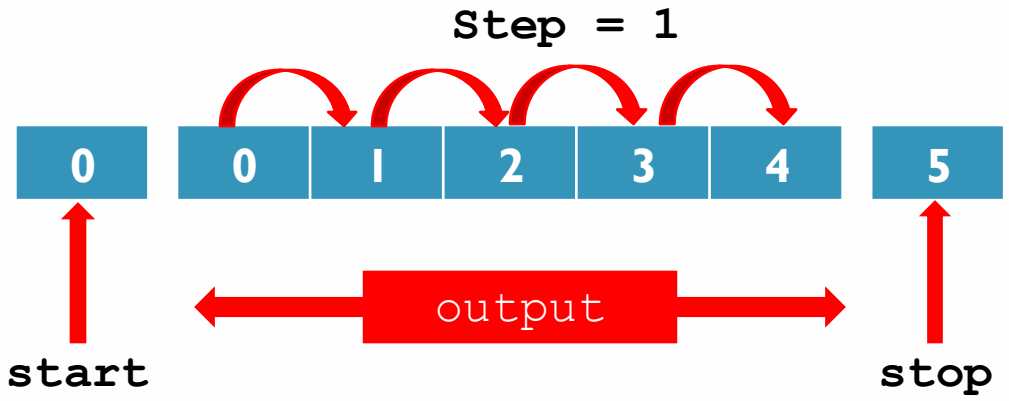
```
for i in range(10):
    if i >= 3:
        print(i)
        continue
        print(i + 1)
print("[for-loop exited!]")
```

THE RANGE FUNCTION (REVISIT)

- do something `n` times: `range(start, stop, step)`
- generates the integer numbers between the given `start` integer to the `stop` integer
- default values are `start = 0` and `step = 1`
- loop until value is `stop - 1`

using only one argument (stop)

```
for i in range(5):
    print(i)
```



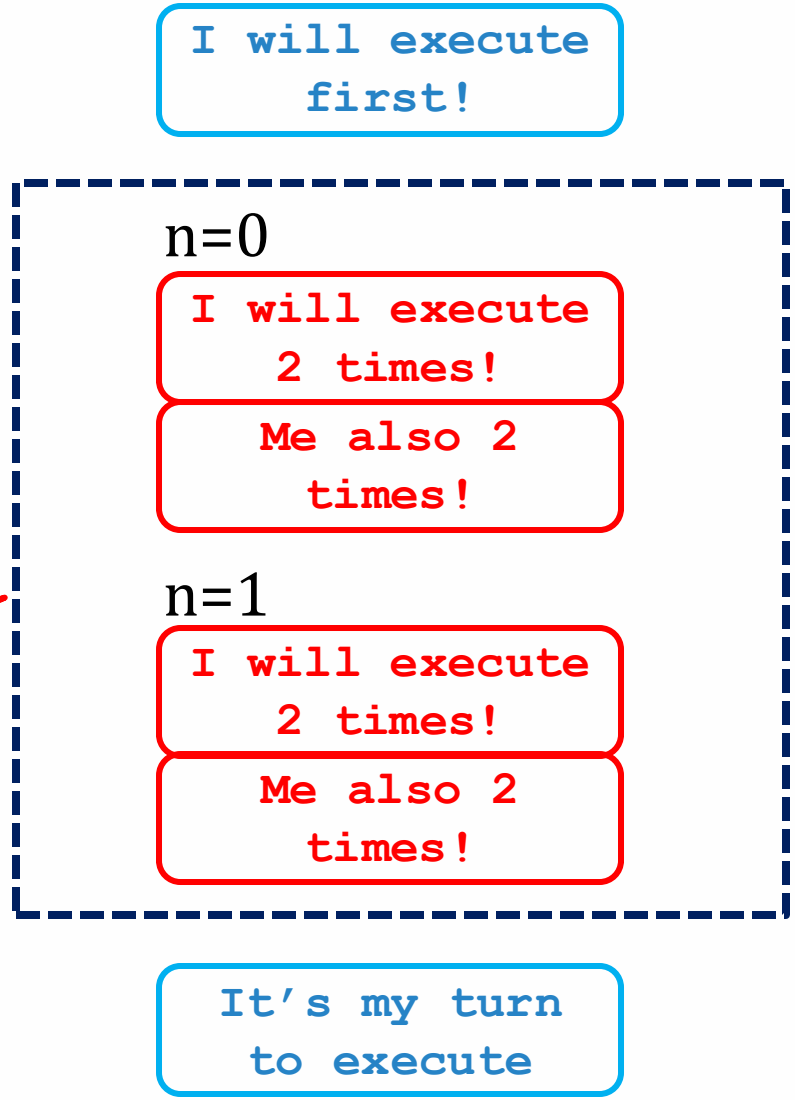
THE RANGE FUNCTION – EXAMPLE 1

```
print("I will execute first!")
for n in range(2):
    print("I will execute 2 times!")
    print("Me also 2 times!")
print("It's my turn to execute!")
```

Loop variable

```
I will execute first!
I will execute 2 times!
Me also 2 times!
I will execute 2 times!
Me also 2 times!
It's my turn to execute!
```

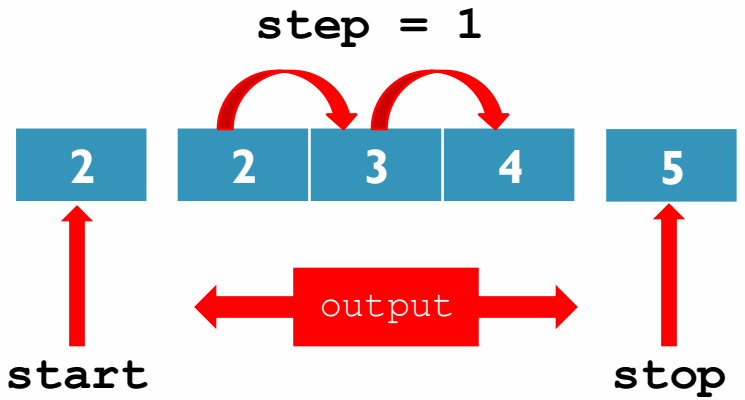
Iteration



THE RANGE FUNCTION – EXAMPLE 2

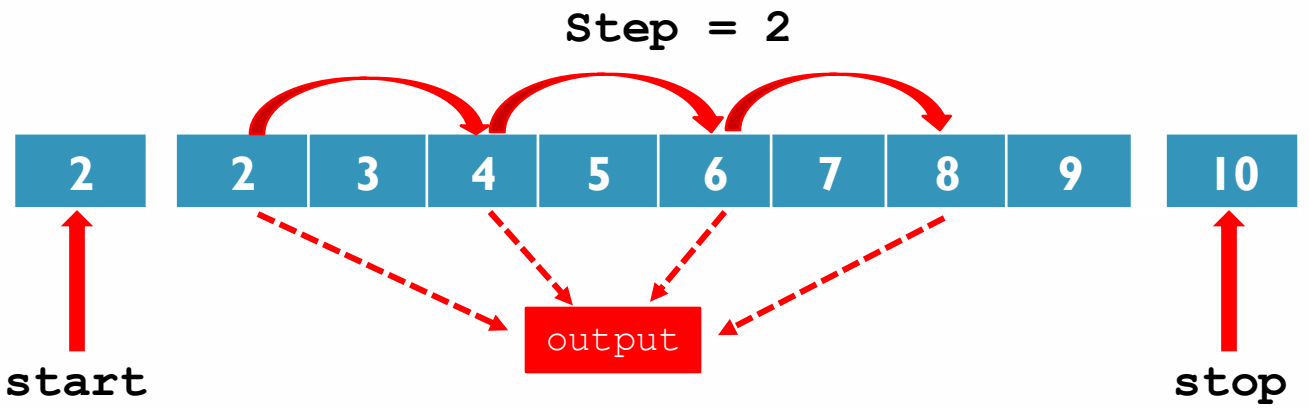
using two arguments (start & stop)

```
for i in range(2,5):
    print(i)
```



using all three arguments

```
for x in range(2, 10, 2):
    print(x)
```

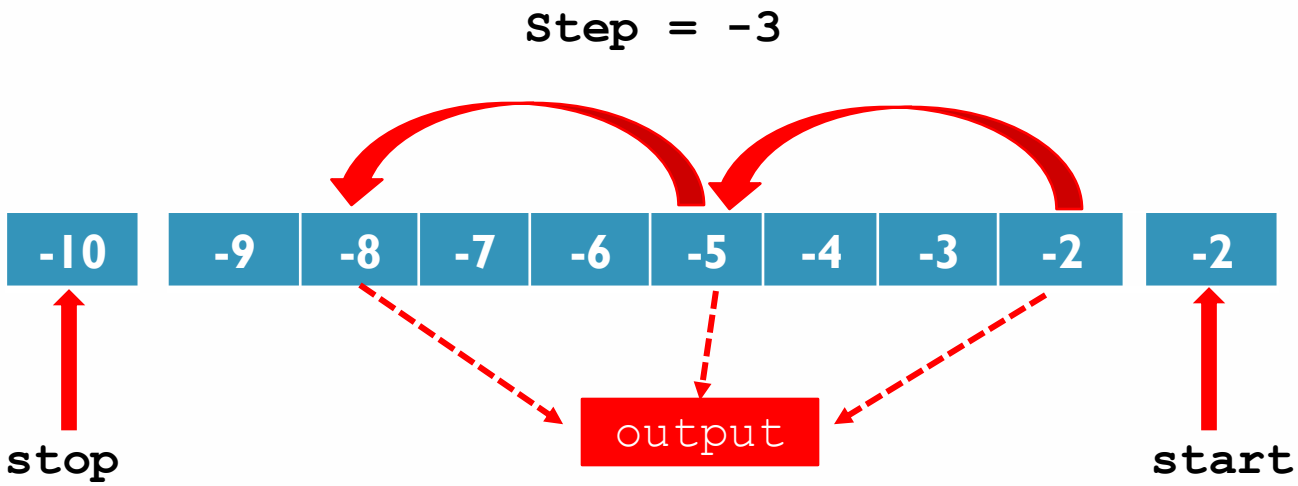


THE RANGE FUNCTION – EXAMPLE 3

using negative step

```
for x in range(-2, -10, -3):
    print(x)
```

to use a negative step, `start` must be a larger value than `stop`

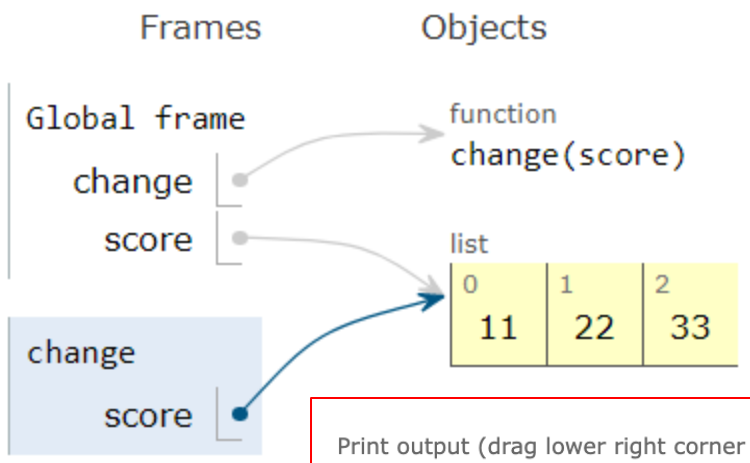


CHANGING THE CONTENTS OF A LIST

```
def change(score):
    for i in range(len(score)):
        score[i] = score[i] + 10

score = [11, 22, 33]
change(score)
print(score)
```

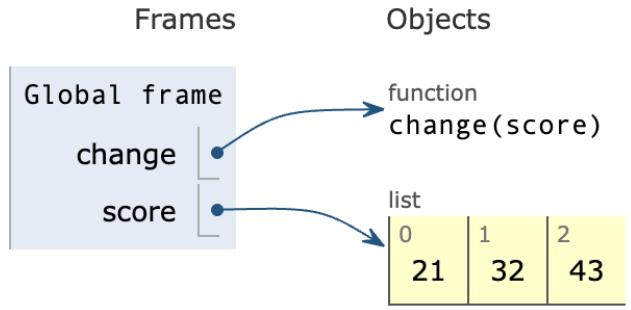
increase 10 for each element in the list



[Demo]

Print output (drag lower right corner to resize)

```
[21, 32, 43]
```



CLASS PRACTICE

Sample Code:

Case 1: [demo]

```
for x in range(20, -10, -13):
    print(x)
    x = 10
```

Case 2: [demo]

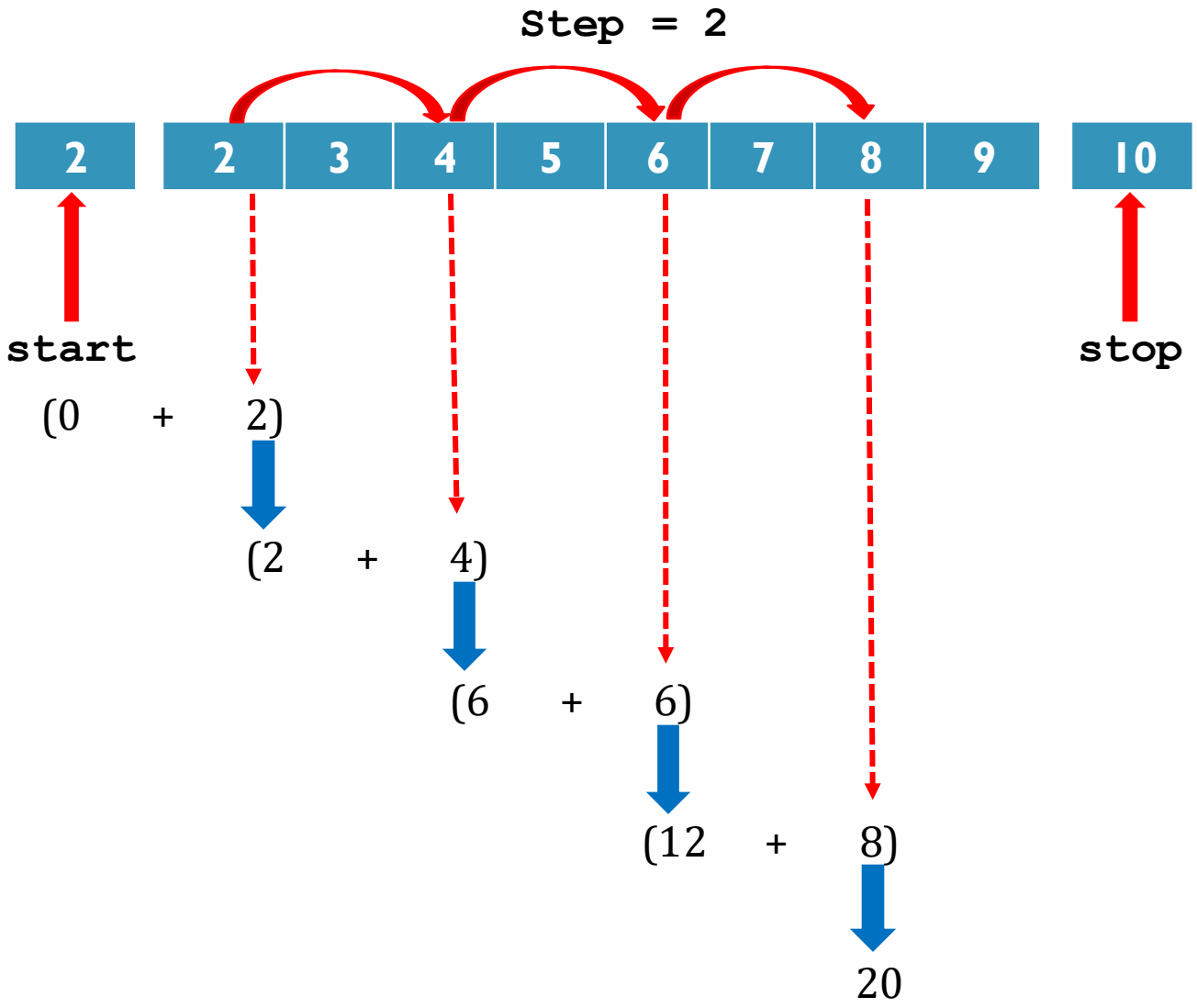
```
for i in range(-2, -15, -5):
    print(i)
    i = 2
print(i)
```

Output:

SUMMING THE ELEMENTS IN RANGE()

accumulator

```
sum = 0
for i in range(2, 10, 2):
    sum += i
print(sum)
```



FOR-LOOP: PROCESSING STRINGS

- Straightforward technique:

```
str = "mellon"
for char in str:
    print(char)
```

- Python range function:

```
str = "mellon"
for index in range(len(str)):
    print(str[index])
```

SUMMING ELEMENTS IN A STRING (1/2)

- Straightforward technique:

```
number = "1234567890"
total = 0
for n in number:
    total += int(n)
print(f"Total is {total}")
```

Convert string to int

- Python range function:

SUMMING ELEMENTS IN A STRING (2/2)

- Straightforward technique:

```
s = "1.233, 4.46, 3.122, 5"
sum = 0
for n in s.split(", "):
    print(n)
    sum += float(n)
print(f"Total is {sum}")
```

Convert string to float

- Python range function:

CLASS PRACTICE

Sample Code:

```
str = "COVID"
for i in str:
    print(      ?      )
```

Output:

```
C
O
V
I
D
```

FOR-LOOP AND CONDITIONS

- Example 1

```
for index in range(len(x)):
    if x[index] == "m":
        print("You have m")
```

- Example 2

```
msg = "Hello World"
i =0
char = "l"
for index in range(len(msg)):
    if msg[index] == char:
        i += 1
print(f"I found {i} '{char}' in x")
```

USING ELSE STATEMENT WITH FOR LOOP

Python allows the use of else condition with for loops.

- Example 1

```
for i in range(5):
    print(i)
else:
    print("Executed else")
```

- Example 2

```
for i in range(5):
    print(i)
    if(i == 3):
        break
else:
    print("Executed else")
```

CLASS PRACTICE

Sample Code:

```
def find_me(x):
    key = "t"
    for
        
    else:
        
```

Output:

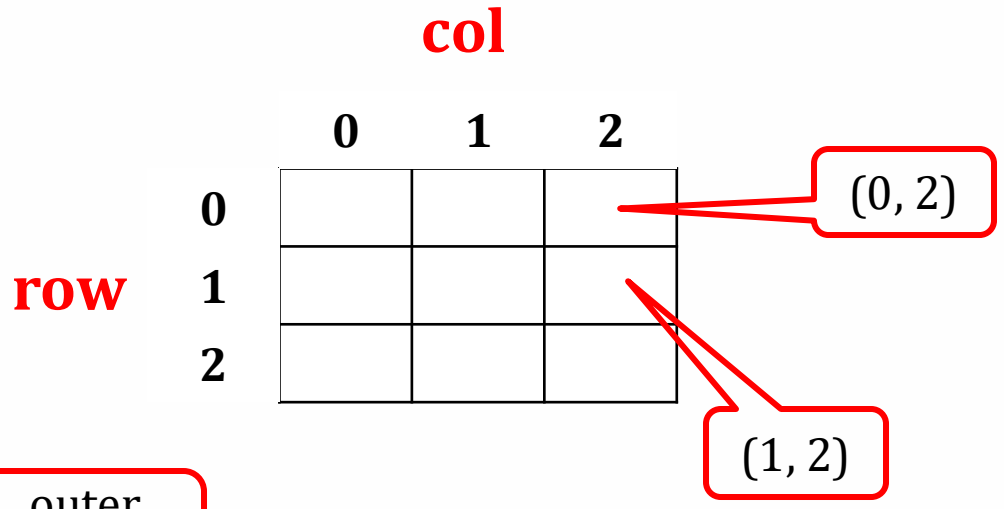
```
find_me("Ali Baba")
find_me("Harry Potter")
```

```
[t] not found in "Ali Baba"
I found [t] in "Harry Potter"
```

NESTED FOR-LOOP

NESTED FOR-LOOP

- Nested loop: **a loop inside a loop.**
- Print every position of a given tic-tac-toe board of 3x3



Steps:

1. Create a variable to hold the rows.
2. Create a second variable for columns.
3. Use a for-loop to iterate through rows.
4. Use the second for-loop to iterate through columns.

outer for-loop

inner for-loop

```
for row in range(3):
    for col in range(3):
        print(f"({row},{col})")
```

NESTED FOR-LOOP EXAMPLE 1

- Find the sum of the following two matrices:

$$M = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad N = \begin{pmatrix} 4 & 3 \\ 2 & 1 \end{pmatrix}$$

$$M + N = \begin{pmatrix} 1+4 & 2+3 \\ 3+2 & 4+1 \end{pmatrix}$$

$$= \begin{pmatrix} 5 & 5 \\ 5 & 5 \end{pmatrix}$$

```
M = [[1, 2], [3, 4]]
N = [[4, 3], [2, 1]]

for i in range(len(M)):
    for j in range(len(M[0])):
        res = M[i][j]+N[i][j]
        print(res, end=" ")
    print("")
```

NESTED FOR-LOOP EXAMPLE 2

Case 1

```
x = 3
for i in range(x):
    for j in range(x):
        print (i * j)
        break
        x = 2
```

Case 2

```
x = 3
for i in range(x):
    for j in range(x):
        print (i * j)
        #break
        x = 2
```

Changing the value of x inside the loop does not affect the number of iterations.

NESTED FOR-LOOP EXAMPLE 3

Case 1

```
x = [1, 2, 3]
y = 0
for i in range(0, 3):
    y += 10
    for j in range(0, 3):
        x[i] += y
print(x)
```

Case 2

```
x = [2, 4, 5]
y = 0
for i in range(0, 3):
    y += 10
    for j in range(0, 3):
        x[j] += y
print(x)
```

NESTED FOR-LOOP EXAMPLE 4

```
for i in range(1, 9):
    for j in range(i):
        print("*", end = " ")
    print()
```

NESTED FOR-LOOP EXAMPLE 5

Write a Python program using nested for loops to generate all unique pairs `(i, j)` where `i` and `j` are from 1 to 5, and `i` is not equal to `j`.

```
(1, 2)
(1, 3)
(1, 4)
(1, 5)
(2, 1)
(2, 3)
(2, 4)
(2, 5)
(3, 1)
(3, 2)
(3, 4)
(3, 5)
(4, 1)
(4, 2)
(4, 3)
(4, 5)
(5, 1)
(5, 2)
(5, 3)
(5, 4)
```

NESTED FOR-LOOP EXAMPLE 6

Write a Python program using nested for loops to generate the following multiplication tables:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9

2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18

3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27

4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45

6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63

8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72

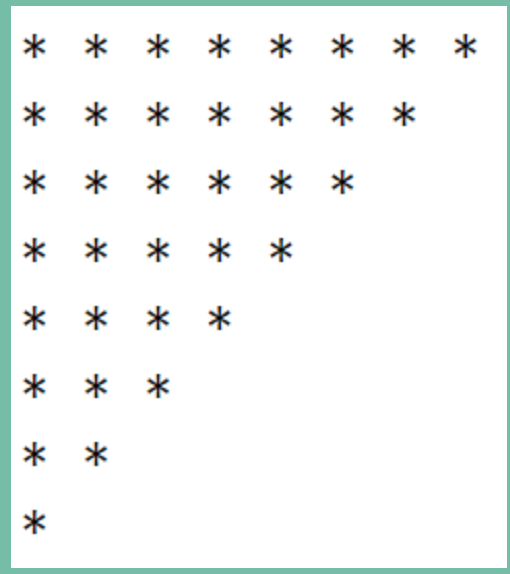
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
```

CLASS PRACTICE

Sample Code:

```
Use a nested for-loop
to print the output:
```

Output:

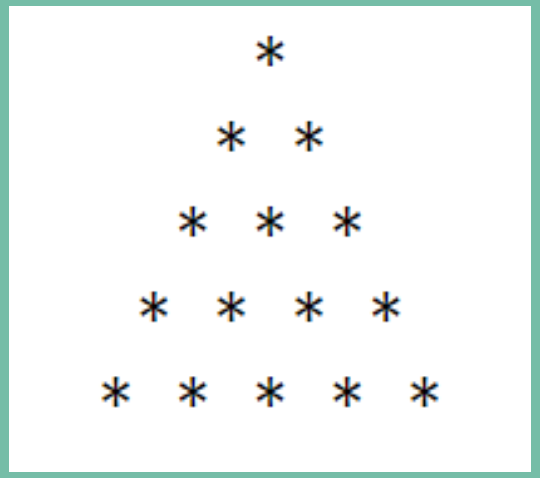


CLASS PRACTICE

Sample Code:

```
Use a nested for-loop
to print the output:
```

Output:



NESTED FOR-LOOP AND CONDITIONS

```
s1 = "harry potter"
s2 = "i love him"
for char1 in s1:
    for char2 in s2:
        if char1 == char2:
            print("common letter is ", end="")
            print(char1)
            break # observe this line
```